

3.3_apply

*apply family

Imagine you have a list of vectors and you want to apply the same function(s) to each element of the list:

- I apply the function manually, selecting the elements
- a **for** loop that iterates over the elements of the list and applies the function(s)

...

```
1 my_list=list(  
2     var1=rnorm(n = 100, mean = 0, sd = 1),  
3     var2=runif(n = 100, min = 0, max = 1),  
4     var3=rnorm(n = 100, mean = 0, sd = 1),  
5     var4=rnorm(n = 100, mean = 0, sd = 1)  
6 )
```

*apply family

Let's compute mean, median and std Result

```
1 # initialize the vectors with arbitrary numbers
2 # but as many as we need
3 means = 1:length(my_list)
4 medians = 1:length(my_list)
5 stds = 1:length(my_list)
6
7 # Loop
8 for(i in 1:length(my_list)){
9
10     means[i] = mean(my_list[[i]])
11     medians[i] = median(my_list[[i]])
12     stds[i] = sd(my_list[[i]])
13 }
```

1 means

```
[1]  0.02129545  0.50867894
0.14344570 -0.07548262
```

1 medians

```
[1] -0.09638977  0.56797256
0.17528729  0.09096798
```

1 stds

```
[1] 1.0427910 0.2835114
1.0393801 0.9386691
```

***apply family**

Everything works! but:

- writing the for loop is laborious, with indices both for the list and for the vector we are filling
- we need to pre-allocate variables (for speed reasons)
- 8 lines of code (for this simple example)

*apply family

R has a family of apply functions such as **lapply**, **sapply**, etc. that let us get the same result in a more concise, fast and simple way:

```
1 means = sapply(my_list, mean)
2 medians = sapply(my_list, median)
3 stds = sapply(my_list, sd)
4
5 means
```

var1	var2	var3	var4
0.02129545	0.50867894	0.14344570	-0.07548262

```
1 medians
```

var1	var2	var3	var4
-0.09638977	0.56797256	0.17528729	0.09096798

```
1 stds
```

var1	var2	var3	var4
1.0427910	0.2835114	1.0393801	0.9386691

***apply** family

`apply( < list > , < function > )`

- What can the **list** be?
 - a list
 - a dataframe
 - a vector
- What can the **function** be?
 - base or imported from a package
 - custom
 - anonymous

*apply family

Before analysing the ***apply** family, I think a further parallel with the for loop we have seen is useful. **apply** is nothing more than a slightly simplified **for** loop.

for loop

```
1 vec = 1:5
2 for(i in vec){
3   print(i)}
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

sapply

```
1 vec = 1:5
2 res = sapply(vec, print)
```

```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

*apply family - custom function

We can also use functions we created ourselves:

```
1 center_var = function(x){  
2   x - mean(x)  
3 }  
4  
5 my_list = list(  
6   vec1 = runif(n = 10, min = 0, max = 1),  
7   vec2 = runif(n = 10, min = 0, max = 1),  
8   vec3 = runif(n = 10, min = 0, max = 1)  
9 )  
10  
11 lapply(my_list, center_var)
```

\$vec1

```
[1] 0.49071715 -0.12017639 -0.07186302 -0.18170562 0.41862526 -0.33412030  
[7] -0.14520500 0.26538832 0.01222864 -0.33388904
```

\$vec2

```
[1] 0.37914252 0.27958712 -0.31489022 -0.21795755 0.29936198 0.16313099  
[7] -0.17477309 0.16149843 -0.54046642 -0.03463376
```

\$vec3

[1] -0.22741742 -0.34336380 -0.32232047 0.20260994 0.56156150 0.01079647

[7] 0.43766024 -0.29823907 0.12227877 -0.14356616

*apply family - implicit vs. explicit

So, just as the **for** loop written as `i in vec` assigns to `i` one **element** of the `vec` object at a time, internally the ***apply** functions take the **first element** of the **input** object (list) and directly **apply** the function we have chosen.

implicit **sapply**

```
1 vec = 1:5  
2 res = sapply(vec, print)
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

explicit **sapply**

```
1 vec = 1:5  
2 res = sapply(vec, function(i) print(i))
```

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

***apply** family - anonymous function

An anonymous function is a function that is not saved in an object but written to be **executed directly**, inside other functions that allow it:

```
1 lapply(my_list, function(x) x - mean(x))
```

\$vec1

```
[1] 0.49071715 -0.12017639 -0.07186302 -0.18170562 0.41862526 -0.33412030  
[7] -0.14520500 0.26538832 0.01222864 -0.33388904
```

\$vec2

```
[1] 0.37914252 0.27958712 -0.31489022 -0.21795755 0.29936198 0.16313099  
[7] -0.17477309 0.16149843 -0.54046642 -0.03463376
```

\$vec3

```
[1] -0.22741742 -0.34336380 -0.32232047 0.20260994 0.56156150 0.01079647  
[7] 0.43766024 -0.29823907 0.12227877 -0.14356616
```

All the types of `*apply`

- `lapply()`: the basic function
- `sapply()`: simplified-apply
- `tapply()`: rarely used, useful with factors
- `apply()`: useful for dataframes/matrices
- `mapply()`: multivariate version, uses several lists at the same time
- `vapply()`: used inside functions and packages

lapply

lapply stands for list-apply and always returns a list, applying the function to each element of the input list:

```
1 res = lapply(my_list, mean)
2 res
```

```
$vec1
[1] 0.3913664
```

```
$vec2
[1] 0.6047078
```

```
$vec3
[1] 0.3667684
```

```
1 class(res)
```

```
[1] "list"
```

sapply

sapply stands for simplified-apply and (tries to) return a simpler version of a list, applying the function to each element of the input list:

```
1 res = sapply(my_list, mean)
2 res
```

```
      vec1      vec2      vec3
0.3913664 0.6047078 0.3667684
```

```
1 class(res)
```

```
[1] "numeric"
```

apply

apply works specifically on dataframes or matrices, applying a function to the rows or the columns:

```
1 my_df = data.frame(x1 = runif(n = 5,min = 1,max = 10),  
2                     x2 = runif(n = 5,min = 3,max = 4))  
3 my_df
```

	x1	x2
1	7.301822	3.357646
2	6.386981	3.752869
3	8.613446	3.306182
4	3.302756	3.870616
5	5.508340	3.535919

apply

Apply the mean function to all rows (1):

```
1 apply(my_df, MARGIN = 1, FUN = mean)
```

```
[1] 5.329734 5.069925 5.959814 3.586686 4.522129
```

Apply the mean function to all columns (2):

```
1 apply(my_df, MARGIN = 2, FUN = mean)
```

```
      x1      x2  
6.222669 3.564647
```

tapply

tapply lets us apply a function to a vector, splitting the vector according to a categorical variable:

```
1 vec = rnorm(75)
2
3 index = rep(c("a", "b", "c"), each = 25)
4
5 tapply(vec, INDEX = index, mean)
```

a	b	c
-0.21304399	0.02248281	0.05038133

Here *INDEX* is a character vector or a factor.

tapply

In this case we compute the mean for each age category:

```
1 my_df = readr::read_csv("data/mydf_2.csv")
2 head(my_df)
```

```
# A tibble: 6 × 4
```

	id	age	age_cat	age_z
	<dbl>	<dbl>	<chr>	<dbl>
1	1	47	adult	1.39
2	2	50	adult	1.67
3	3	23	young	-0.774
4	4	39	adult	0.672
5	5	28	young	-0.322
6	6	15	adolescent	-1.50

```
1 tapply(my_df$age, my_df$age_cat, mean)
```

adolescent	adult	young
16.6	40.5	24.0

vapply

vapply is a more robust version of the previous ones from a programming point of view. In practice it lets (and requires) us to specify in advance the type of data we expect as a result:

```
1 vapply(my_list, FUN = mean, FUN.VALUE = numeric(length = 1))
```

vec1	vec2	vec3
0.3913664	0.6047078	0.3667684

FUN.VALUE = numeric(length = 1): indicates that each result is a single numeric value.

mapply

mapply lets us handle several lists at the same time for more complex scenarios. For example, we want to use the **rnorm()** function to generate 4 samples combining different means and standard deviations:

```
1 mus = list(10, 20, 30, 40)
2 stds = list(1, 2, 3, 4)
3
4 mapply(function(x,y) rnorm(n = 5, mean = x, sd = y),
5         mus, stds, SIMPLIFY = FALSE)
```

```
[[1]]
```

```
[1] 10.625957 10.889984 10.024787  9.246613  9.391001
```

```
[[2]]
```

```
[1] 19.22194 15.83161 20.22224 18.65874 17.34550
```

```
[[3]]
```

```
[1] 28.48536 28.33813 29.93324 32.68209 29.18278
```

```
[[4]]
```

```
[1] 42.25903 40.19806 47.91491 35.46042 40.21208
```

IMPORTANT: all the lists included must have the same length!

mapply

`mapply(function(x,y) rnorm(n = 5, mean = x, sd = y), mus, stds, SIMPLIFY = FALSE)`

- **function(...)**: an anonymous function, as we saw before, which can have n elements
- **rnorm(n = 5, mean = x, sd = y)**: the actual anonymous function, where we have the placeholders x and y
- **mus, stds**: the lists corresponding, **in order**, to the placeholders, so x = mus and y = stds
- **SIMPLIFY = FALSE**: simply says to return a list and not to try (like sapply) to simplify the result

mapply as a for

The same result (in a more verbose way) can be obtained with a **for** loop, using the iterator *i* several times:

```
1 mus = list(10, 20, 30)
2 stds = list(1,2,3)
3 res = vector(mode = "list", length = length(mus)) # empty list
4
5 for(i in 1:length(mus)){
6   res[[i]] = rnorm(n = 6, mean = mus[[i]], sd = stds[[i]])
7 }
8 res
```

```
[[1]]
```

```
[1]  9.408912  9.945890 10.423683  7.877252  9.379916  8.869178
```

```
[[2]]
```

```
[1] 20.21967 21.07804 18.07781 21.70890 20.17084 20.96706
```

```
[[3]]
```

```
[1] 30.13637 28.55157 35.06225 26.71362 30.95603 30.86664
```

replicate

This function lets us repeat an operation n times, without using an iterator or a placeholder.

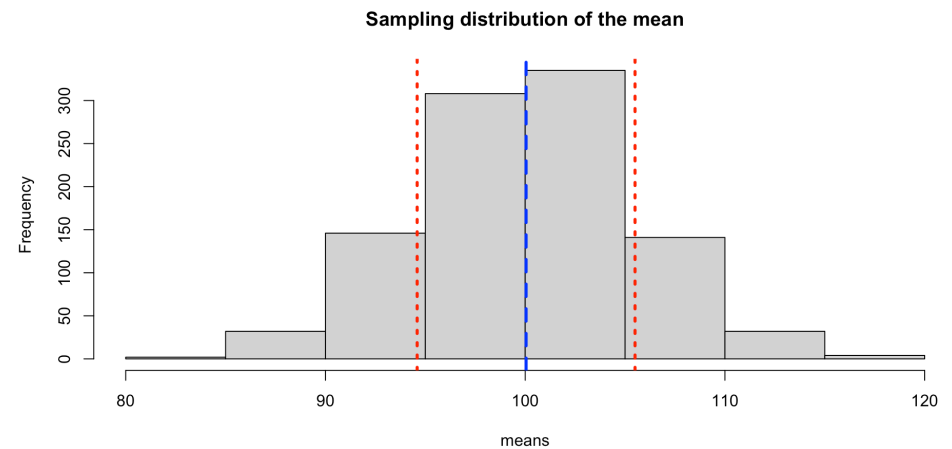
replicate(n , $expr$)

- n is the number of repetitions
- $expr$ is the portion of code to repeat

replicate

We sample 1000 times from a normal distribution and compute the mean, AKA the sampling distribution of the mean

```
1  nrep=1000
2
3  nsample=30
4
5  mu=100
6
7  sd=30
8
9  means=
10  replicate(
11    n = nrep,
12    expr = {mean(
13      rnorm(nsample, mu, sd))}
14  )
```



Now let's practice!

Open this link and keep it open:

<https://etherpad.wikimedia.org/p/arca-corsoR>